

CRITÉRIOS DE AVALIAÇÃO DE LINGUAGENS DE PROGRAMAÇÃO

PROF. MARCELO LADEIRA CIC/UNB

BASEADO NO CAPÍTULO 1 DO LIVRO TEXTO DO SEBESTA.

INTRODUÇÃO

1.1 Razões para cursar LP

1.2 Domínios de programação

1.3 Influências sobre o projeto da linguagem

1.4 Paradigmas de linguagens

1.5 Critérios de avaliação da linguagem

RAZÕES PARA CURSAR LP

1. Aumentar a capacidade para expressar conceitos
A LP impõe limites às formas de algoritmos
Pode-se simular facilidades inexistentes ...
2. Melhorar a capacidade de escolher LP adequadas
“Quando só se tem martelo, tudo parece prego”
As LP mais populares nem sempre são melhores
3. Facilitar aprender novas linguagens
O domínio dos conceitos das LP facilita entender como eles são incorporados ao projeto de uma LP
4. Entender melhor a importância da implementação
Melhora o uso da LP utilizada
Ex.: acesso a matrizes (armazenadas por linha ou por coluna?), common em Fortran (problema na manutenção), uso de recursão, alocação de struct em C

program main	subroutine sub1 (argumentos)
real alpha, beta	real x, y
common /coeff/ alpha, beta	integer n
	common /coeff/ x,y
5. Aumentar a capacidade de projeto de novas LP

```
long* ptr = (long*)malloc(10 * sizeof(long));
```

DOMÍNIOS TÍPICOS DE PROGRAMAÇÃO

O domínio da aplicação força o projeto da LP no sentido de atender as suas especificidades

1. Aplicações Científicas (início nos anos 40)

Estruturas de dados simples: arranjos (arrays).
Grande número de cálculos aritméticos.
Estruturas de controle: laços iterativos e seleções
Ex.: Fortran

2. Aplicações Comerciais (início nos anos 50)

Facilidades para produzir relatórios elaborados
Precisão ao descrever e armazenar números decimais e dados de caracteres.
Capacidade de especificar aritmética decimal.
Ex. COBOL, planilhas e banco de dados

12345 -> 1 2 3 4 5 BCD

78 -> 7 8 BCD

1 2 4 2 3 BCD

3. Inteligência Artificial

Início: manipulação simbólica e não numérica.
Estrutura de dados: listas encadeadas.
Capacidade de criar e executar código runtime.
Ex. LISP, Prolog.

DOMÍNIOS TÍPICOS DE PROGRAMAÇÃO (CONTINUAÇÃO)

4. Programação de Sistemas

Software básico: SO e ferramentas de suporte à programação de um computador

Eficiência de execução: rapidez.

Recursos de baixo nível para interface com dispositivos externos.

Ex.: C, C++ e Assembly

5. Linguagens de Scripting

Permitem a criação de um arquivo com uma lista de comandos a serem executados.

Ex. csh (C shell), ksh (korn shell) e Perl

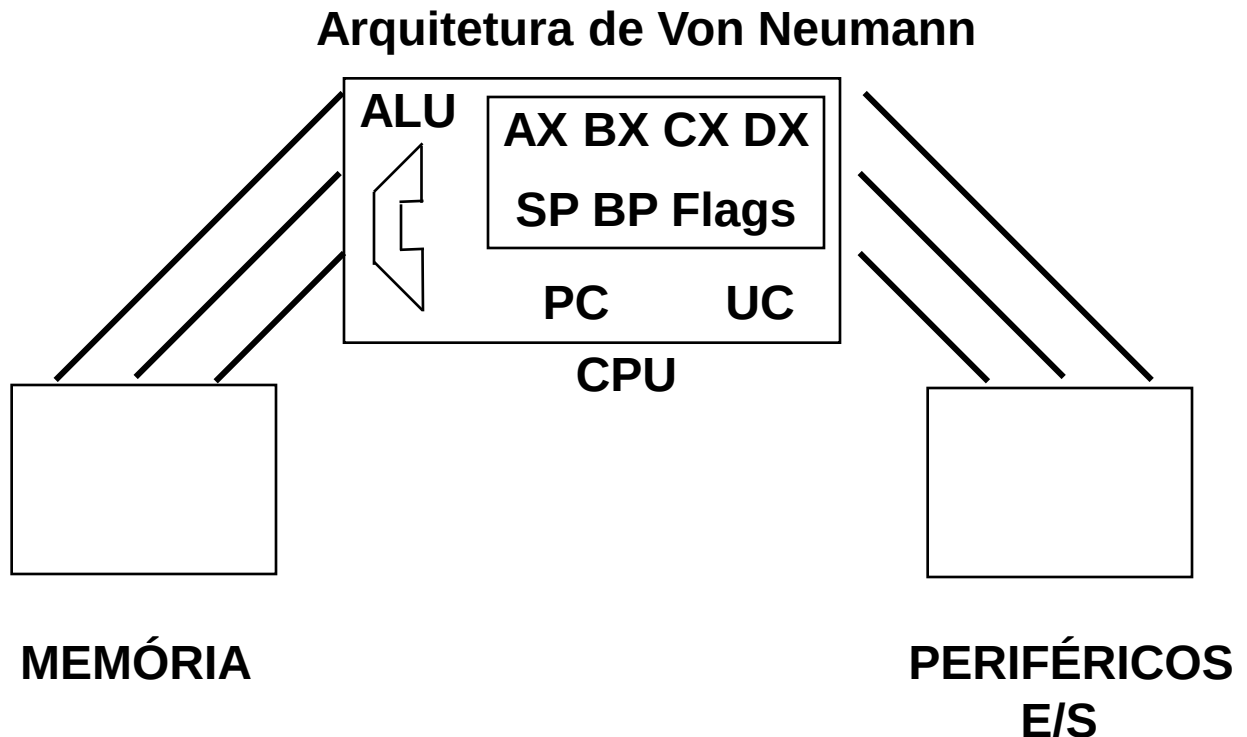
6. Linguagens de Propósito Especiais

Características e aplicabilidade muito específicas torna difícil compara-las com outras.

Ex. SAS, SPSS, Spark

INFLUÊNCIAS SOBRE O PROJETO DA LINGUAGEM

1. Arquitetura do Computador



- **CPU**

- unidade lógica aritmética (ALU), unidade de controle (UC), registradores (propósito geral, apontadores e flags), contador de programa (PC)

- **Barramentos**

- dados, endereço, controle

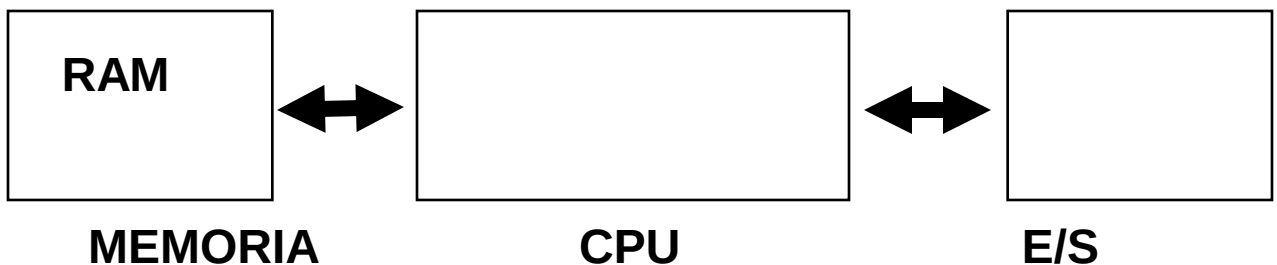
- **Periféricos de entrada/saída mapeados na memória**

- RAM – volátil e capacitiva

- ROM – não volátil e física (ponte de silício)

INFLUÊNCIAS SOBRE O PROJETO DA LINGUAGEM

“Usamos linguagens imperativas porque usamos a arquitetura de Von Neumann”



- Dados e programas armazenados na memória
- Ciclo busca-decodificação-busca de operandos-execução (fetch-execute cycle)
- Processamento é seqüencial (**localidade de referência**)
- Execução rápida porque são armazenadas em posições adjacentes da memória
- **Efeito do desvio incondicional (goto) ?**
- Essa rapidez desencoraja o uso da recursão, o qual é básico em linguagens funcionais
- Gargalo: velocidade da CPU >> velocidade de transferência da memória para CPU >> velocidade de transferência de dados dos periféricos para a CPU

INFLUÊNCIAS SOBRE O PROJETO DA LINGUAGEM

- Técnicas para redução do gargalo de von Neumann

- diferença de velocidade entre a CPU e memória de acesso direto:

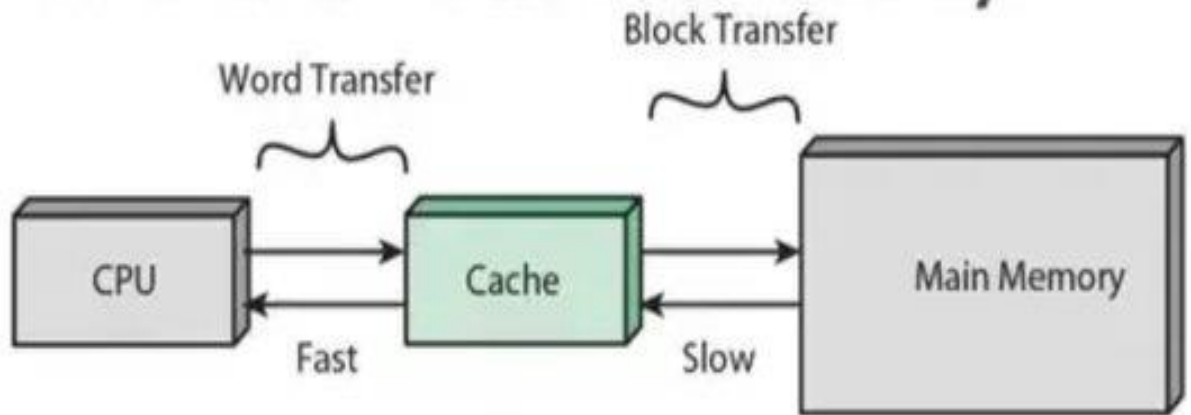
- Memória cache (localidade de referência!)
- Processadores com pipelining (estágios) NOP (bolha)

- diferença de velocidade entre a memória de acesso direto (RAM) e os dispositivos de entrada/saída:

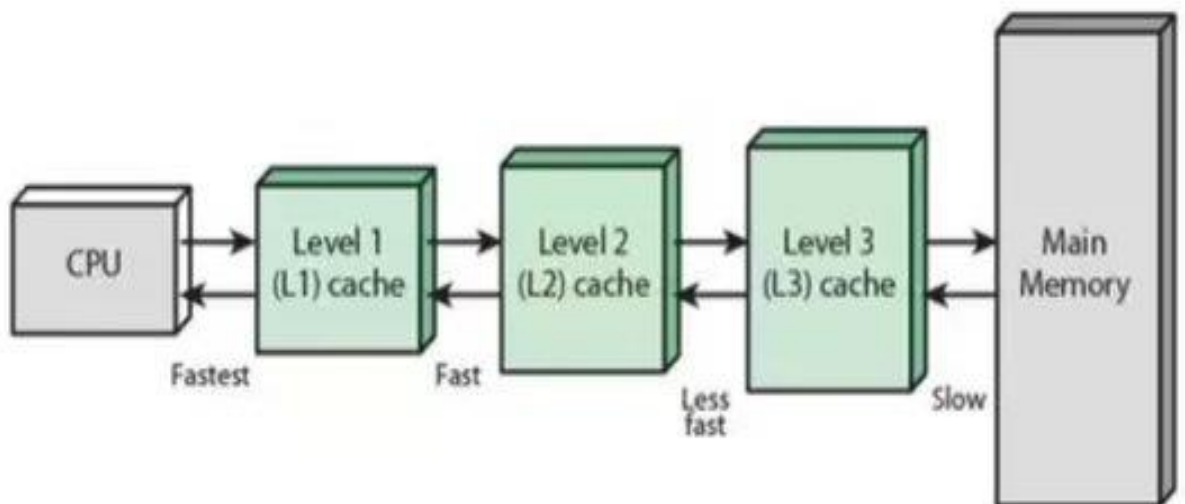
- blocagem e bufferização
- DMA (Acesso Direto à Memória)
- canais de controle de dispositivos de entrada/saída

ARQUITETURA DE MEMÓRIA

Cache and Main Memory



(a) Single cache



normalmente tem 64k a 256k no total !

FONTE: WWW.QUORA.COM

INFLUÊNCIAS SOBRE O PROJETO DA LINGUAGEM

2. Metodologias de Programação

Preocupação migrou de maximizar uso do hardware para maximizar a produtividade e manutenabilidade.

- *Anos 50 e início dos anos 60*
 - Aplicações simples
 - Preocupação: eficiência do uso do hardware
- *Final dos anos 60 (programação estruturada)*
 - Pessoas tornam-se importantes
 - Preocupação: legibilidade e melhores estruturas de controle
 - Projeto top-down e refinamento sucessivo
- *Final dos anos 70*
 - Abstração de dados
 - Preocupação: mudança da orientação para processos para a orientação para dados
- *Meados dos anos 80 (Orientação a objetos: reuso)*
 - Abstração de dados, encapsulamento e herança

PARADIGMAS* DE LINGUAGEM

1. Imperativas

- algoritmo é especificado com detalhes
- a ordem de execução dos comandos é incluída
- possui comando de atribuição

2. Funcionais (ou aplicativas)

- principal meio de fazer computações é aplicando funções a determinados parâmetros
- não possui comando de atribuição
- utiliza recursão extensamente
- fluxo de execução é determinado pelas funções

3. Lógicas

- regras especificadas sem ordem particular
- implementação escolhe a ordem de execução
- fluxo de execução é determinado pelas regras

4. Orientadas a Objetos

- abstração de dados encapsula processamento com objetos de dados e oculta o acesso a eles
- herança e vinculação de tipos em tempo de execução

CRITÉRIOS DE AVALIAÇÃO DA LINGUAGEM

1. Legibilidade

O critério mais importante: facilita a manutenção

- *Simplicidade global*

- Muitos componentes básicos: ruim
- Multiplicidade de recursos (mais de uma maneira de realizar 1 operação particular): ruim
- Sobrecarga: boa mas pode ser mal usada
- Muita simplicidade (falta estrutura): ruim
- *Ortogonalidade* (toda combinação possível de conceitos primitivos é legal e significativa)

- Significado é independente do contexto
- Torna a LP fácil de aprender e de ler
- Muita ortogonalidade: maior complexidade

- *Instruções de controle*

- Ex.: goto em Fortran e assign em PL/1: ruim

- *Tipos de dados e estruturas*

- Facilidades adequadas para defini-los: bom
- Ex.: tipos booleanos, registros e listas

- *Aspectos da sintaxe*

- identificadores pequenos: ruim
- palavras especiais (delimitadores): bom
- forma e semântica compatível: bom

CRITÉRIOS DE AVALIAÇÃO DA LINGUAGEM

2. Capacidade de Escrita

Medida da facilidade de se usar uma LP para criar programas em um certo domínio.

- *Simplicidade e ortogonalidade*

- Poucos componentes básicos e regras consistentes para combiná-las: bom

- Muita ortogonalidade: difícil depuração

- *Suporte para abstração*

- Capacidade de definir/usar estruturas ou operações complicadas, abstraindo os detalhes, facilita a escrita de programas.

- abstração de processo: subprogramas

- abstração de dados: classes, registros, etc.

- *Expressividade*

- Existência de formas convenientes de especificar computações facilita a escrita.

- *Sintaxe* (restrita pode reduzir a capacidade de escrita)

- delimitadores e operadores poderosos: bom

CRITÉRIOS DE AVALIAÇÃO DA LINGUAGEM

3. Confiabilidade

Programa é confiável se comporta de acordo com as especificações em todas as situações.

- *Verificação de tipos*

- Tempo de compilação (desejável) ou execução
“Quando mais cedo se descobrir um erro, melhor”

- *Tratamento de exceções*

- Capacidade de interceptar erros na execução, pôr em prática medidas corretivas e prosseguir.

- Ex.: on <condição> do <comando>

- *Aliasing*

- Ex.: Membros de união e ponteiros definidos para apontar para a mesma posição de memória: perigoso.

- *Legibilidade e capacidade de escrita*

- Quanto mais fácil for escrever um programa, mais provável que ele seja correto, mais fácil entender e mais fácil mantê-lo.

CRITÉRIOS DE AVALIAÇÃO DA LINGUAGEM

4. Custo

Custo final de 1 LP é função de suas características.

- *Custo de treinamento*
 - simplicidade e ortogonalidade da LP
 - experiência dos programadores
- *Custo para escrever programas +*
 - capacidade de escrita e adequação a aplicação particular
- *Custo para compilar programas*
 - nível de otimização do código: compromisso tamanho do código X velocidade de execução(-)
- *custo para executar programas*
 - projeto da LP. Ex.: verificação de tipos.
- *Custo do sistema de implementação da linguagem*
- *Custo da má confiabilidade em sistemas críticos +*
- *Custo da manutenção dos programas + (2 a 4x)*
 - legibilidade

CRITÉRIOS DE AVALIAÇÃO DA LINGUAGEM

5. Outros

- *Portabilidade*

Facilidade com que programas podem ser mudados de uma implementação para outra

- **Grau de padronização da LP**

- *Generalidade*

Aplicação a uma ampla faixa de aplicações

- *Qualidade da Definição*

- Qualidade do documento oficial que define a LP

Tem ou não gramática!!!

Existência de Conflitos em critérios

- **Projetistas:** elegância e generalidade
- **Implementadores:** dificuldade de implementar as construções e os recursos da LP
- **Programadores:** capacidade de escrita e legibilidade